1.Algorithm: Implementation of Lexical Analyzer Using Lex Tool

1. Start the program.

2. Include the required header file `stdio.h`.

3. Define the patterns for letters and digits using Lex definitions.

4. Specify the rules to recognize the following tokens:

- Keywords such as `int`, `float`, `if`, `while` and `return`.
- Identifiers consisting of letters, digits and underscores, beginning with a letter or underscore.
- Floating-point constants consisting of digits, a decimal point and digits.
- Integer constants consisting of one or more digits.
- Operators such as `+`, `-`, `=`, `==`, `>=` and `&&`.
- Special symbols such as `()`, `{}`, `;` and `,`.

5. Ignore whitespace characters such as spaces, tabs and newlines.

6. If any character does not match the defined patterns, display it as an unknown token.

7. Define the `yywrap()` function to return `1` when the input ends.

8. In the `main()` function, display a message asking the user to enter C code and call `yylex()` to start lexical analysis.

9. The Lex analyzer scans the input, matches lexemes with the defined patterns and displays their corresponding token categories.

10. Stop the program when the end of input is reached (Ctrl+D).

2.Algorithm: Count the Number of Lines, Words and Characters Using Lex

1. Start the program.

2. Include the required header file `stdio.h` and initialize the counters `lines`, `words` and `chars` to zero.

3. Define the Lex rules to recognize newline characters, whitespace and words.

4. If a newline character (`\n`) is encountered, increment the line counter and character counter by one.

5. If spaces or tabs are encountered, increment the character counter by the length of the matched text (`yyleng`).

6. If a sequence of non-whitespace characters is encountered, increment the word counter by one and increase the character counter by its length (`yyleng`).

7. Define the `yywrap()` function to return `1` when the input ends.

8. In the `main()` function, display a message asking the user to enter text and call `yylex()` to begin scanning.

9. Continue scanning until the end of input is reached (Ctrl+D).

10. Display the total number of lines, words and characters.

11. Stop the program.

3.Algorithm: Replace the Substring `abc` with `ABC` Using Lex

1. Start the program.

2. Include the required header file `stdio.h`.

3. Define a Lex rule to recognize the substring `abc`.

4. Whenever the substring `abc` is encountered, replace it with `ABC` and print it.

5. For all other characters, including newline characters, use `ECHO` to print them unchanged.

6. Define the `yywrap()` function to return `1` when the input ends.

7. In the `main()` function, display a message asking the user to enter a string and call `yylex()` to start scanning.

8. Continue scanning the input until the end of input is reached (Ctrl+D).

9. Stop the program.

4.Algorithm: Count Vowels and Consonants Using Lex

1. Start the program.
2. Include the header file `stdio.h` and initialize the vowel and consonant counters to zero.
3. Define a Lex rule to recognize vowels (`a, e, i, o, u` in both uppercase and lowercase).
4. Whenever a vowel is encountered, increment the vowel counter by one.

5. Define a Lex rule to recognize alphabetic characters that are not vowels as consonants.
6. Whenever a consonant is encountered, increment the consonant counter by one.
7. Ignore all other characters, including spaces, digits and special symbols.
8. Define the `yywrap()` function to return `1` when the input ends.
9. In the `main()` function, prompt the user to enter a string and call `yylex()` to start scanning.
10. Continue scanning until the end of input (Ctrl+D).
11. Display the total number of vowels and consonants.
12. Stop the program.

5.Algorithm: Recognize a Valid Arithmetic Expression Using YACC

1. Start.

2. Create a Lex program to recognize numbers, arithmetic operators (`+`, `-`, `*`, `/`), parentheses and newline characters.

3. Include the YACC-generated header file in the Lex program to access the token definitions.

4. Define a regular expression to recognize integers and convert them into numeric values using `atoi()`. Return the `NUMBER` token to YACC.

5. Ignore spaces and tabs, and return the recognized operators and parentheses as individual tokens.

6. Create a YACC program and declare the token `NUMBER` and the required functions.

7. Specify the precedence and associativity of operators:

- `+` and `-` have lower precedence and are left-associative.
- `*` and `/` have higher precedence and are left-associative.
- Unary minus is assigned the highest precedence using `UMINUS`.

8. Define grammar production rules to recognize:

- Numbers.
- Addition, subtraction, multiplication and division.
- Unary minus.
- Expressions enclosed in parentheses.

9. Define the starting production to accept a valid arithmetic expression followed by a newline.

10. Define the error-handling function to display a syntax error message when the input is invalid.

11. In the main function, display a prompt and call `yyparse()` to begin syntax analysis.

12. Compile the Lex and YACC programs, link the required libraries and execute the generated program.

13. Enter an arithmetic expression. If it satisfies the grammar, display "Valid Expression"; otherwise, display a syntax error.

14. Stop.

6.Algorithm: Implementation of Calculator Using LEX and YACC

Step 1: Start.

Step 2: Create a Lex program (`calc.l`) to recognize the input tokens.

1. Include the YACC-generated header file `y.tab.h`.
2. Recognize integers using the regular expression `[0-9]+`.
3. Convert the recognized number into an integer using `atoi()` and return the `NUMBER` token.
4. Ignore spaces and tabs.
5. Return newline characters and arithmetic operators or parentheses to YACC.

Step 3: Create a YACC program (`calc.y`) to evaluate arithmetic expressions.

1. Include the required header files and declare `yylex()` and `yyerror()`.
2. Declare the token `NUMBER`.
3. Define operator precedence and associativity:
    - `+` and `-` have lower precedence.
    - `*` and `/` have higher precedence.
    - Unary minus has the highest precedence using `UMINUS`.
4. Define grammar rules for numbers, addition, subtraction, multiplication, division, unary minus and parenthesized expressions.
5. Use semantic actions to perform the corresponding arithmetic operations and store the result in `$$`.
6. Check for division by zero and report an error if the divisor is zero.
7. Define the input and line rules to accept multiple expressions, each ending with a newline.

Step 4: Define the error-handling function `yyerror()` to display error messages.

Step 5: In the main function, display a prompt and call `yyparse()` to begin parsing and evaluation.

Step 6: Compile the Lex and YACC programs, link the required libraries and execute the calculator.

Step 7: Enter arithmetic expressions. Evaluate each valid expression according to operator precedence and display its result.

Step 8: Stop.